Objective:
Establish a secure, encrypted VPN tunnel between two Google Cloud Compute Engine instances using WireGuard. Using WireGuard and GCP official Documentation.

Demonstrate configuration, connectivity verification, and troubleshooting of a peer-to-peer VPN connection, with gateway routing for Internet access through one node.

Establish a secure WireGuard VPN tunnel between two GCP VMs (**`wg-a`** and **`wg-b`**) and enable `wg-a` to act as a gateway for Internet access.

Goals:

- Understand WireGuard's peer-based VPN configuration
- Practice GCP instance creation, firewall setup, and Linux networking
- Implement and verify secure connectivity between cloud hosts

---

Tools Used:

- Google Cloud Platform: Compute Engine, VPC Firewall rules
- Ubuntu 22.04 LTS on both VMs
- WireGuard
- Linux networking tools: **`ping`**, **`tcpdump`**, **`sysctl`**

Linux utilities:
`wg, wg-quick, ip, iptables, sysctl, ping, curl, tcpdump`

Command-line tools:
`gcloud, ssh, vim, ufw`

---

1. Create VMs on GCP

Created 2 VMS using GCP CLI w/ ip forwarding capabilities(wg-a & wg-b)

```
alihammam0178@cloudshell:~ (projects-477315)$ gcloud compute instances create wg-a \
> --zone=us-central1-a \
> --machine-type=e2-medium \
> --tags=http-server,https-server \
> --image-family=debian-11 \
> --image-project=debian-cloud
```

```
alihammam0178@cloudshell:~ (projects-477315)$ gcloud compute instances create wg-b \
> --zone=us-central1-a \
> --machine-type=e2-medium \
> --tags=http-server,https-server \
> --image-family=debian-11 \
> --image-project=debian-cloud
```

List view

```
alihammam0178@cloudshell:~ (projects-477315)$ gcloud compute instances list
NAME: wg-a
ZONE: us-central1-a
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
INTERNAL_IP: 10.128.0.2
EXTERNAL_IP: 35.224.177.202
STATUS: RUNNING

NAME: wg-b
ZONE: us-central1-a
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
INTERNAL_IP: 10.128.0.3
EXTERNAL_IP: 34.133.128.56
STATUS: RUNNING
alihammam0178@cloudshell:~ (projects-477315)$
```

2. Created firewall rule to allow UDP 51820

```
alihammam0178@cloudshell:~ (projects-477315)$ gcloud compute firewall-rules create allow-wireguard \
  --network=default \
  --allow=udp:51820 \
  --source-ranges=0.0.0.0/0 \
  --target-tags=wireguard-vm \
  --description="Allow WireGuard VPN traffic on UDP 51820"
```

3. Install WireGuard on both VM's and generated keys

```
alihammam0178@wg-a:~$ sudo apt update && sudo apt install wireguard -y
Hit:1 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:5 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3073 kB]
Get:6 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.0 kB]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [2808 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 c-n-f Metadata [13.9 kB]
Fetched 6298 kB in 2s (3033 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
13 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wireguard is already the newest version (1.0.20210914-1ubuntu2).
0 upgraded, 0 newly installed, 0 to remove and 13 not upgraded.
```

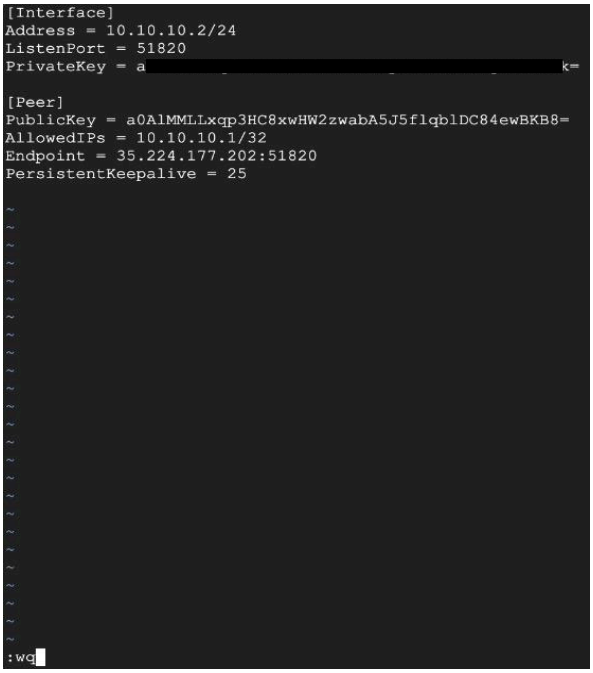
Updated packages and installed wireguard on wg-a and wg-b

```
root@wg-a:/etc/wireguard# wg genkey | tee privatekey | wg pubkey > publickey
```

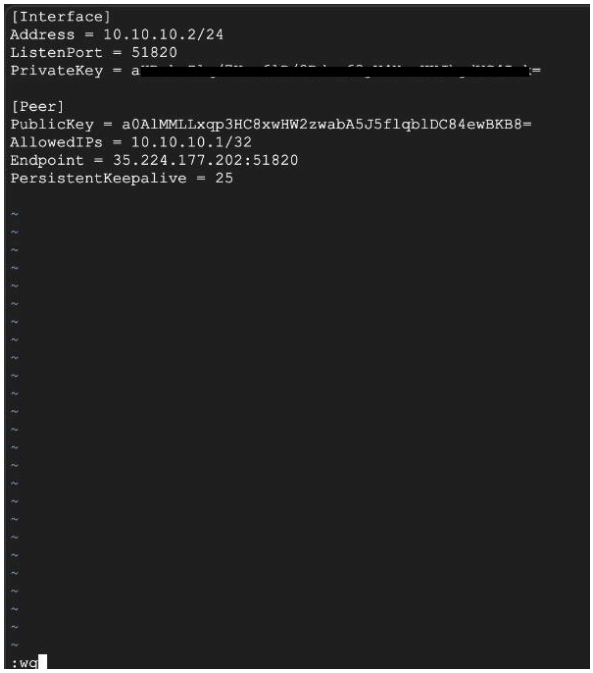
```
alihammam0178@wg-a:~$ sudo -i
root@wg-a:~# cd /etc/wireguard
root@wg-a:/etc/wireguard# ls
privatekey  publickey  wg0.conf
```

Generated Keys and sorted. Created WireGuard tunnel configuration. Set umask 077 to enforce a private-by-default security posture.

Configured `wg0.conf` on both VMs



-wg-a wg0.conf



-wg-b wg0.conf

4. Bring up WireGuard on both VM's

```
root@wg-a:/etc/wireguard# wg-quick up wg0
wg-quick: `wg0' already exists
root@wg-a:/etc/wireguard# systemctl enable wg-quick@wg0
Created symlink /etc/systemd/system/multi-user.target.wants/wg-quick@wg0.service → /lib/systemd/system/wg-quick@.servi
ce.
root@wg-a:/etc/wireguard# wg show
interface: wg0
  public key: a0AlMMLLxqp3HC8xwHW2zwabA5J5flqblDC84ewBKB8=
  private key: (hidden)
  listening port: 51820

peer: yYG9mTy2+iL8EvJlVBLlFHpr/yJNB4vGDZT5XnXjCDo=
  endpoint: 34.133.128.56:51820
  allowed ips: 10.10.10.2/32
  latest handshake: 1 minute, 58 seconds ago
  transfer: 540 B received, 5.20 KiB sent
  persistent keepalive: every 25 seconds
```

```
root@wg-b:/etc/wireguard# wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.10.10.2/24 dev wg0
[#] ip link set mtu 1380 up dev wg0
root@wg-b:/etc/wireguard# sudo systemctl enable wg-quick@wg0
Created symlink /etc/systemd/system/multi-user.target.wants/wg-quick@wg0.service → /lib/systemd/system/wg-quick@.service.
root@wg-b:/etc/wireguard# wg show
interface: wg0
  public key: yYG9mTy2+iL8EvJlVBLlFHpr/yJNB4vGDZT5XnXjCDo=
  private key: (hidden)
  listening port: 51820

peer: a0AlMMLLxqp3HC8xwHW2zwabA5J5flqblDC84ewBKB8=
  endpoint: 35.224.177.202:51820
  allowed ips: 10.10.10.1/32
  latest handshake: 36 seconds ago
  transfer: 156 B received, 180 B sent
  persistent keepalive: every 25 seconds
root@wg-b:/etc/wireguard#
```

Used wg-quick up & wg show to confirm the handshake between both VM's

Used sudo systemctl enable wg-quick@wg0 to ensure encryption on startup

5. Verified connectivity

```
root@wg-a:/etc/wireguard# ping -c 10 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
64 bytes from 10.10.10.2: icmp_seq=1 ttl=64 time=1.79 ms
64 bytes from 10.10.10.2: icmp_seq=2 ttl=64 time=0.942 ms
64 bytes from 10.10.10.2: icmp_seq=3 ttl=64 time=1.22 ms
64 bytes from 10.10.10.2: icmp_seq=4 ttl=64 time=1.02 ms
64 bytes from 10.10.10.2: icmp_seq=5 ttl=64 time=1.27 ms
64 bytes from 10.10.10.2: icmp_seq=6 ttl=64 time=1.04 ms
64 bytes from 10.10.10.2: icmp_seq=7 ttl=64 time=1.65 ms
64 bytes from 10.10.10.2: icmp_seq=8 ttl=64 time=0.979 ms
64 bytes from 10.10.10.2: icmp_seq=9 ttl=64 time=1.33 ms
64 bytes from 10.10.10.2: icmp_seq=10 ttl=64 time=1.05 ms

--- 10.10.10.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9013ms
```

Pinged wg-b via wg-a

```
root@wg-b:/etc/wireguard# ping -c 10 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=1.68 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=1.04 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=0.993 ms
64 bytes from 10.10.10.1: icmp_seq=4 ttl=64 time=1.13 ms
64 bytes from 10.10.10.1: icmp_seq=5 ttl=64 time=0.998 ms
64 bytes from 10.10.10.1: icmp_seq=6 ttl=64 time=1.12 ms
64 bytes from 10.10.10.1: icmp_seq=7 ttl=64 time=1.02 ms
64 bytes from 10.10.10.1: icmp_seq=8 ttl=64 time=1.23 ms
64 bytes from 10.10.10.1: icmp_seq=9 ttl=64 time=1.24 ms
64 bytes from 10.10.10.1: icmp_seq=10 ttl=64 time=1.03 ms

--- 10.10.10.1 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9011ms
rtt min/avg/max/mdev = 0.993/1.147/1.683/0.198 ms
```

Pinged wg-a via wg-b

Verified connectivity, confirmed handshake and VM's were talking.

```
listening on wg0, link-type RAW (Raw IP), snapshot length 262144 bytes
19:12:27.920716 IP 10.10.10.2 > wg-a: ICMP echo request, id 2, seq 1, length 64
19:12:27.920750 IP wg-a > 10.10.10.2: ICMP echo reply, id 2, seq 1, length 64
19:12:28.922513 IP 10.10.10.2 > wg-a: ICMP echo request, id 2, seq 2, length 64
19:12:28.922536 IP wg-a > 10.10.10.2: ICMP echo reply, id 2, seq 2, length 64
19:12:29.923680 IP 10.10.10.2 > wg-a: ICMP echo request, id 2, seq 3, length 64
19:12:29.923699 IP wg-a > 10.10.10.2: ICMP echo reply, id 2, seq 3, length 64
19:12:30.924965 IP 10.10.10.2 > wg-a: ICMP echo request, id 2, seq 4, length 64
19:12:30.924986 IP wg-a > 10.10.10.2: ICMP echo reply, id 2, seq 4, length 64
19:12:31.926272 IP 10.10.10.2 > wg-a: ICMP echo request, id 2, seq 5, length 64
19:12:31.926293 IP wg-a > 10.10.10.2: ICMP echo reply, id 2, seq 5, length 64
19:12:32.927493 IP 10.10.10.2 > wg-a: ICMP echo request, id 2, seq 6, length 64
19:12:32.927513 IP wg-a > 10.10.10.2: ICMP echo reply, id 2, seq 6, length 64
19:12:33.928216 IP 10.10.10.2 > wg-a: ICMP echo request, id 2, seq 7, length 64
19:12:33.928237 IP wg-a > 10.10.10.2: ICMP echo reply, id 2, seq 7, length 64
19:12:34.929438 IP 10.10.10.2 > wg-a: ICMP echo request, id 2, seq 8, length 64
19:12:34.929467 IP wg-a > 10.10.10.2: ICMP echo reply, id 2, seq 8, length 64
19:12:35.930827 IP 10.10.10.2 > wg-a: ICMP echo request, id 2, seq 9, length 64
19:12:35.930850 IP wg-a > 10.10.10.2: ICMP echo reply, id 2, seq 9, length 64
19:12:36.932286 IP 10.10.10.2 > wg-a: ICMP echo request, id 2, seq 10, length 64
19:12:36.932308 IP wg-a > 10.10.10.2: ICMP echo reply, id 2, seq 10, length 64
^Z
[1]+  Stopped                 tcpdump -i wg0
```

Ran tcpdump from wg-a, shows received packets from wg-b.

6. Enable IP forwarding on wg-a (to act as gateway)

```
alihammam0178@wg-a:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
```

Enabled IP forwarding temporarily before adjusting the configuration

```
# See sysctl.conf (5) for information.
#

#kernel.domainname = example.com

# Uncomment the following to stop low-level messages on console
#kernel.printk = 3 4 1 3

##############################################################
# Functions previously found in netbase
#

# Uncomment the next two lines to enable Spoof protection (reverse-path filter)
# Turn on Source Address Verification in all interfaces to
# prevent some spoofing attacks
#net.ipv4.conf.default.rp_filter=1
#net.ipv4.conf.all.rp_filter=1

# Uncomment the next line to enable TCP/IP SYN cookies
# See http://lwn.net/Articles/277146/
# Note: This may impact IPv6 TCP sessions too
#net.ipv4.tcp_syncookies=1

# Uncomment the next line to enable packet forwarding for IPv4
net.ipv4.ip_forward=1

# Uncomment the next line to enable packet forwarding for IPv6
#  Enabling this option disables Stateless Address Autoconfiguration
#  based on Router Advertisements for this host
#net.ipv6.conf.all.forwarding=1


###################################################################
# Additional settings - these settings can improve the network
# security of the host and prevent against some network attacks
# including spoofing attacks and man in the middle attacks through
-- INSERT --                                                32,1          9%
```

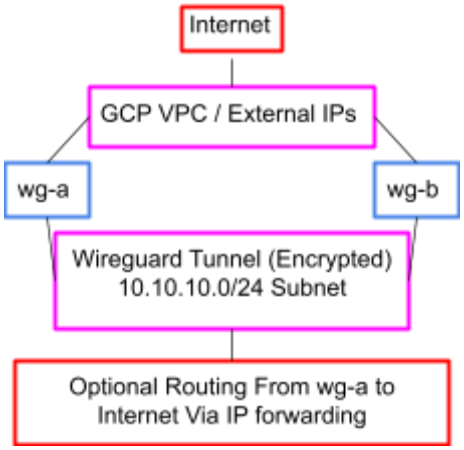
```
root@wg-a:/etc# vim sysctl.conf
root@wg-a:/etc# sysctl -p
net.ipv4.ip_forward = 1
```

Uncommented IP forwarding in sysctl.conf to ensure forwarding on reboot.

sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

Used iptables-persistent tools to ensure rules maintain after reboot

Diagram



Troubleshooting

- Permission denied → created a root shell (sudo -i) to work consistently in an escalated privilege.
- Firewall → ensure UDP 51820 is allowed.
- Ping fails → check allowed IPs and endpoints.
- Ensured iptables rules & port forwarding persistent after reboot.

Result

- WireGuard tunnel successfully established between wg-a and wg-b
- Verified with `ping`, `wg show`, and `tcpdump`
- IP forwarding enabled on wg-a for Internet access through the tunnel
- Project demonstrates secure encrypted VPN and routing between two cloud VMs